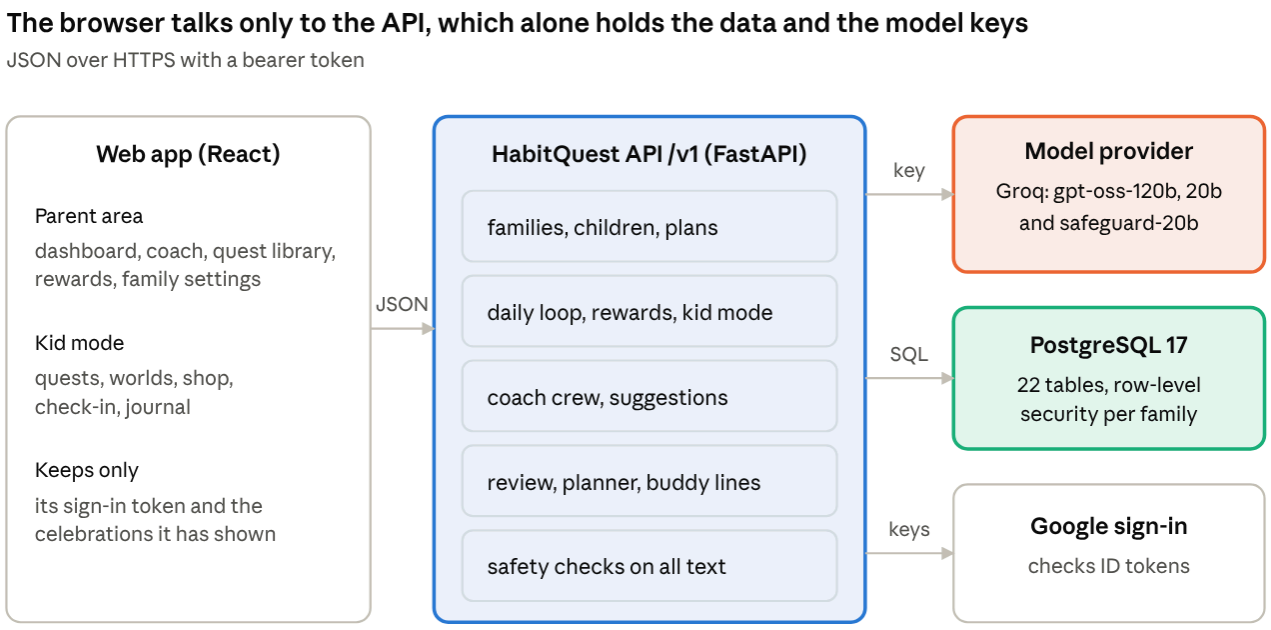
HabbitQuest Engineering & Quality

Nov 3, 2026 · @Rahul

One React app, one FastAPI service and one PostgreSQL database; the database itself keeps each family's data apart, and 452 automated tests run on every change.

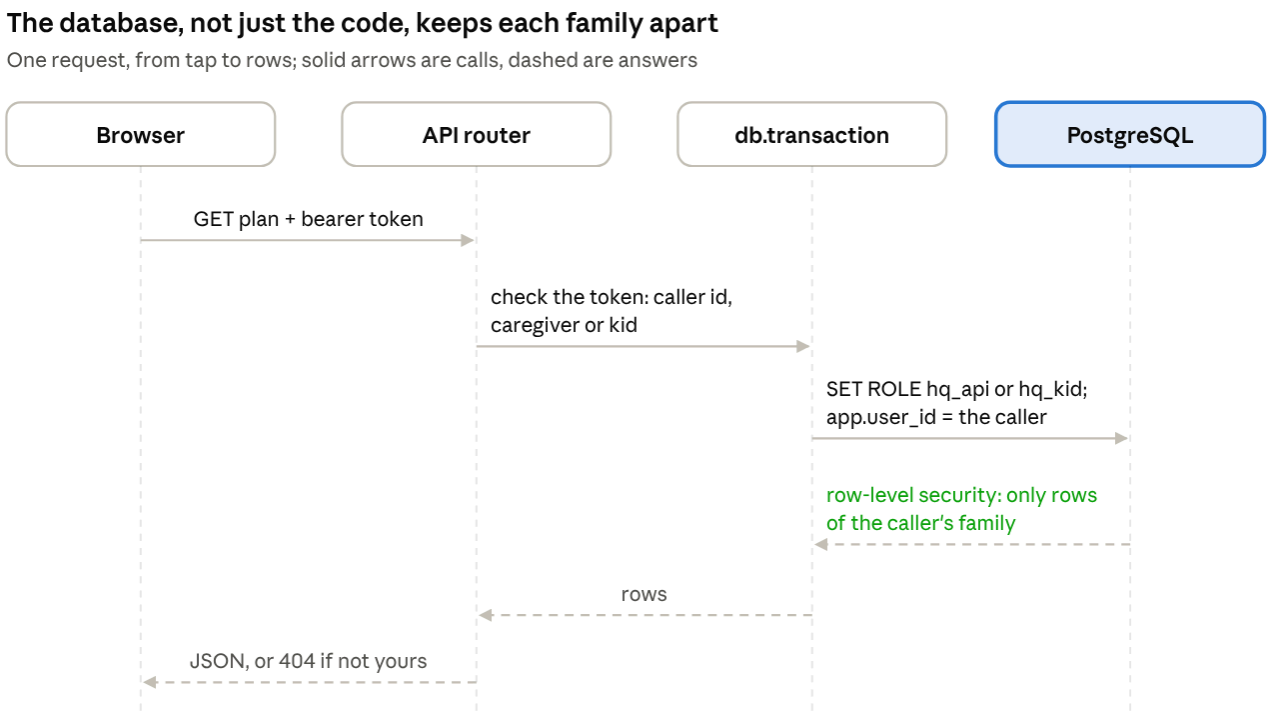
1. Architecture



architecture · browser, API, models, database, Google

Part	Built with	Hosted on
Web app	React 18, TypeScript, Vite	Any static host (local today)
API	Python 3.12, FastAPI, pydantic, LangGraph	Render web service
Database	PostgreSQL 17, row-level security	Render PostgreSQL
AI models	gpt-oss via Groq (OpenAI-compatible)	Groq

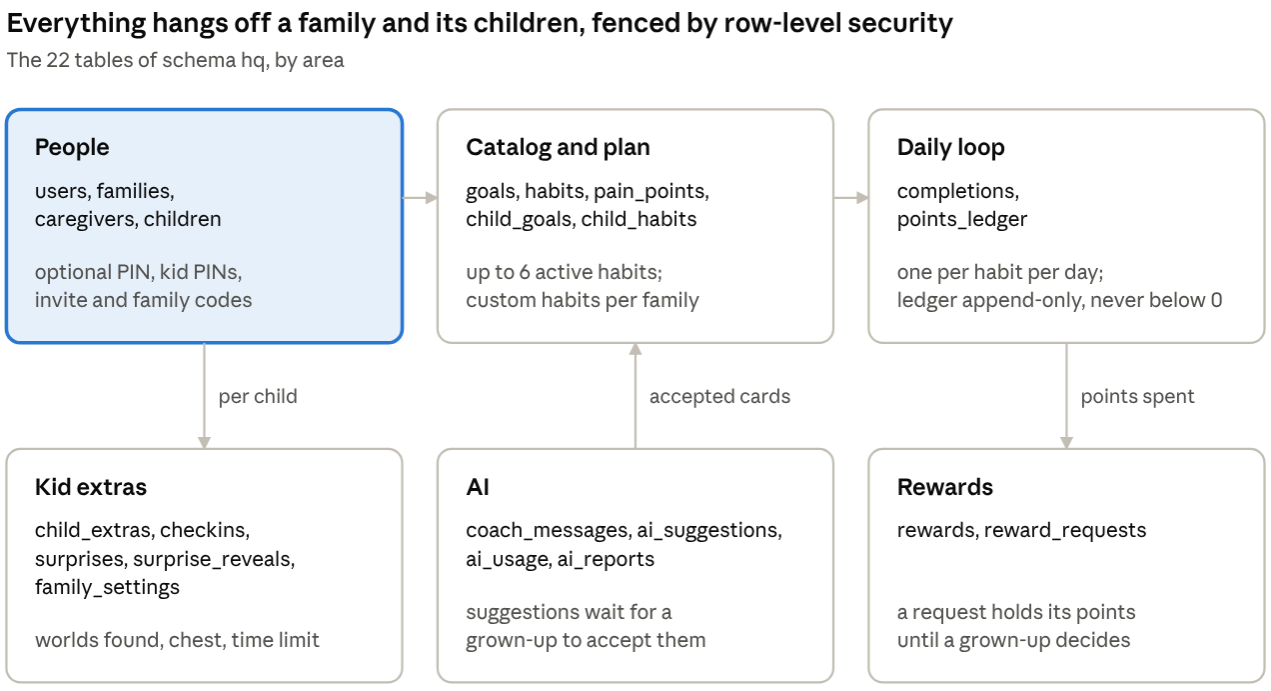
2. How one request stays inside its family



one request · token, transaction, row-level security

Even a query that forgets `WHERE family_id = ...` cannot return another family's rows. Kid tokens use a narrower role that sees one child and can only submit today's quests.

3. Data model



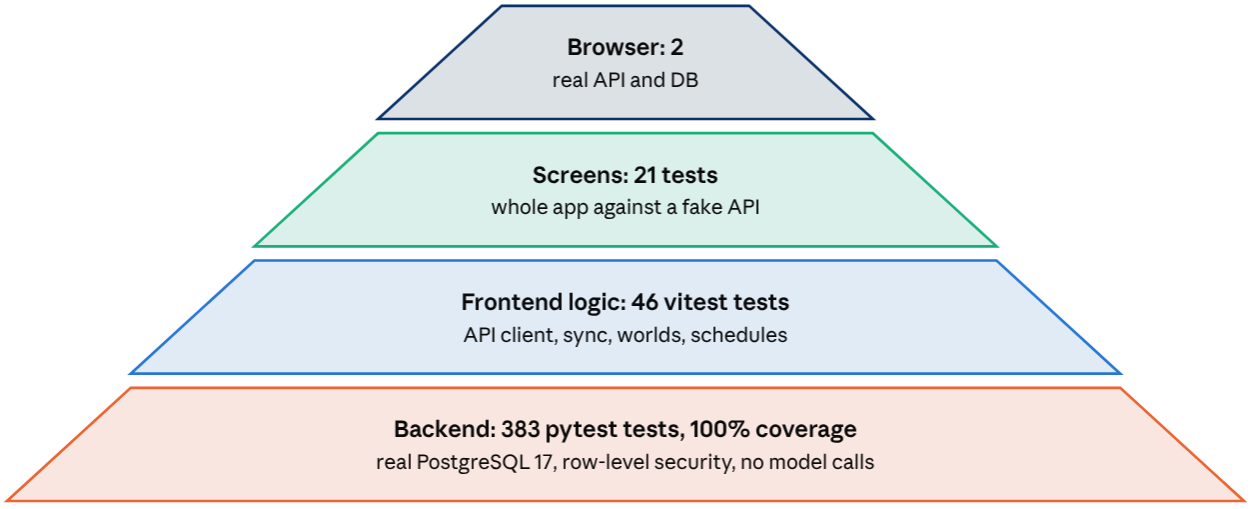
data model · 22 tables in 6 areas

The schema lives in `backend/db/migrations/` and is applied in order on every deploy; `backend/docs/schema.md` has a diagram of every table.

4. Automated tests

Most tests are fast; two browser journeys prove the whole stack

452 automated tests by layer; CI runs every layer on every change



Not in CI: scripts/eval_coach.py scores the coach against 41 realistic parent messages with real models.

test pyramid · 383 backend, 67 frontend, 2 browser

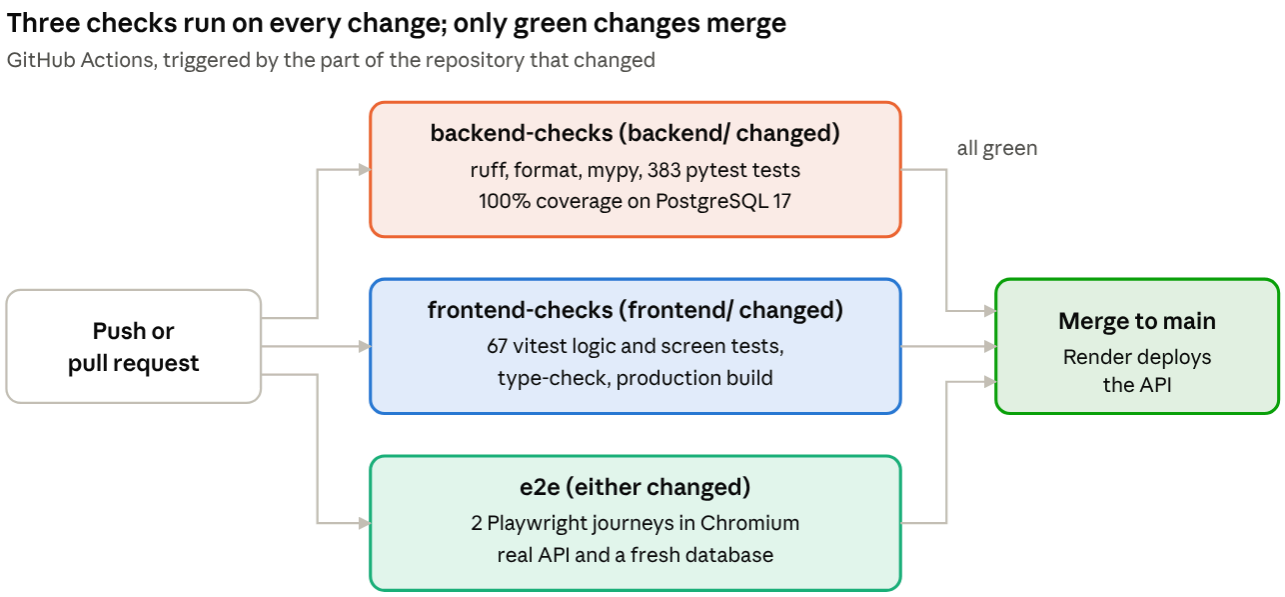
Backend tests by area (301 test functions, 383 cases with parameters):

Area	Tests	What they prove
AI: coach, crew, safety, tracing, guidance	125	Risk checks, fallbacks, schema limits, redaction; no real model is called
Kid features and the journey	77	Rewards, chests, check-ins, surprises, time limits, Sparks and worlds
Accounts and sign-in	36	Google and development login, PINs, kid sign-in, rate limits
Plans and habits	27	Age and tier rules, custom habits, the 6-quest limit
Daily loop and points	19	Approve once, boosts, a balance that never goes below 0
API and schema	17	Row-level security between families, migrations

Frontend: 21 screen tests click through sign-in, approvals with the PIN, the Quest library and kid mode against a fake API; 2 Playwright journeys run a family's first day and the PIN against the real API. A screen test fails if a page calls an endpoint the fake does not know.

Found by tests: the screen tests caught a dashboard bug where a failed approval's reason flashed and vanished; it is fixed.

5. CI on every change



CI · 3 checks before a merge

Each change lands as one pull request; the workflows live in `.github/workflows/` (`test.yml`, `frontend.yml`, `e2e.yml`).

6. Run it locally

About 15 minutes from clone to a running app (Python 3.12 with uv, PostgreSQL 17, Node.js 22).

```
git clone https://github.com/rahulwarekar20/habbit-quest.git
cd habbit-quest/backend && uv sync --locked
uv run python scripts/bootstrap_local.py        # local database and a git-ignored .env
uv run python -m uvicorn habitquest_backend.main:app --reload
cd ../frontend && npm install && npm run dev     # http://localhost:5180
```

The developer guide in `ARCHITECTURE.md` and `backend/docs/localsetup.md` covers the rest; every file in the repository opens with a comment saying what it is for.

7. Before real families use it

- [ ] Paid database with backups, before the free one expires on 28 November 2026

- [ ] Always-on API instance (the free one sleeps when idle)
- [ ] Groq paid tier, and failover to a second provider
- [ ] Host the frontend; add its address to CORS and the Google client
- [ ] Turn off the development login; publish the Google consent screen
- [ ] Verifiable parental consent (COPPA) and a privacy review
- [ ] Monitoring and alerts
- [x] ~~Accounts, sync across devices, several grown-ups~~
- [x] ~~Row-level security, export and deletion~~
- [x] ~~Automated tests in CI at 100% backend coverage~~